Chapter 8

Fitting regression models

Most of this book is devoted to examples and tools for the practical use and understanding of regression models, starting with linear regression with a single predictor and moving to multiple predictors, nonlinear models, and applications in prediction and causal inference. In this chapter we lay out some of the mathematical structure of inference for regression models and some algebra to help understand estimation for linear regression. We also explain the rationale for the use of the Bayesian fitting routine `stan_glm` and its connection to classical linear regression. This chapter thus provides background and motivation for the mathematical and computational tools used in the rest of the book.

8.1 Least squares, maximum likelihood, and Bayesian inference

We now step back and consider *inference*: the steps of estimating the regression model and assessing uncertainty in the fit. We start with *least squares*, which is the most direct approach to estimation, based on finding the values of the coefficients a and b that best fit the data. We then discuss *maximum likelihood*, a more general framework that includes least squares as a special case and to which we return in later chapters when we get to logistic regression and generalized linear models. Then we proceed to *Bayesian inference*, an even more general approach that allows the probabilistic expression of prior information and posterior uncertainty.

Least squares

In the classical linear regression model, $y_i = a + bx_i + \epsilon_i$, the coefficients a and b are estimated so as to minimize the errors ϵ_i. If the number of data points n is greater than 2, it is not generally possible to find a line that gives a perfect fit (that would be $y_i = a + bx_i$, with no error, for all data points $i = 1, \ldots, n$), and the usual estimation goal is to choose the estimate $(\hat{a}, \hat{b})$ that minimizes the sum of the squares of the residuals,

$$r_i = y_i - (\hat{a} + \hat{b}x_i).$$

We distinguish between the *residuals* $r_i = y_i - (\hat{a} + \hat{b}x_i)$ and the *errors* $\epsilon_i = y_i - (a + bx_i)$. The model is written in terms of the errors, but it is the residuals that we can work with: we cannot calculate the errors as to do so would require knowing a and b.

The residual sum of squares is

$$\text{RSS} = \sum_{i=1}^{n} (y_i - (\hat{a} + \hat{b}x_i))^2. \tag{8.1}$$

The $(\hat{a}, \hat{b})$ that minimizes RSS is called the least squares or ordinary least squares or OLS estimate and can be written in matrix notation as,

$$\hat{\beta} = (X^t X)^{-1} X^t y, \tag{8.2}$$

where $\beta = (a, b)$ is the vector of coefficients and $X = (\mathbf{1}, x)$ is the matrix of predictors in the regression. In this notation, $\mathbf{1}$ represents a column of ones—the constant term in the regression—and must be included because we are fitting a model with an intercept as well as a slope. We show more general notation for linear regression with multiple predictors in Figure 10.8.

Expression (8.2) applies to least squares regression with any number of predictors. In the case of regression with just one predictor, we can write the solution as,

$$\hat{b} = \frac{\sum_{i=1}^{n}(x_i - \bar{x})\, y_i}{\sum_{i=1}^{n}(x_i - \bar{x})^2}, \tag{8.3}$$

$$\hat{a} = \bar{y} - \hat{b}\bar{x}. \tag{8.4}$$

We can then can write the least squares line as,

$$y_i = \bar{y} + \hat{b}\,(x_i - \bar{x}) + r_i;$$

thus, the line goes through the mean of the data, $(\bar{x}, \bar{y})$, as illustrated in Figure 6.3.

Formula (8.2) and the special case (8.3)–(8.4) can be directly derived using calculus as the solution to the problem of minimizing the residual sum of squares (8.1). In practice, these computations are done using efficient matrix solution algorithms in R or other software.

Estimation of residual standard deviation σ

In the regression model, the errors ϵ_i come from a distribution with mean 0 and standard deviation σ: the mean is zero by definition (any nonzero mean is absorbed into the intercept, a), and the standard deviation of the errors can be estimated the from data. A natural way to estimate σ would be to simply take the standard deviation of the residuals, $\sqrt{\frac{1}{n}\sum_{i=1}^{n} r_i^2} = \sqrt{\frac{1}{n}\sum_{i=1}^{n}(y_i - (\hat{a} + \hat{b}x_i))^2}$, but this would slightly underestimate σ because of *overfitting*, as the coefficients $\hat{a}$ and $\hat{b}$ have been set based on the data to minimize the sum of squared residuals. The standard correction for this overfitting is to replace n by $n - 2$ in the denominator (with the subtraction of 2 coming from the estimation of two coefficients in the model, the intercept and the slope); thus,

$$\hat{\sigma} = \sqrt{\frac{1}{n-2}\sum_{i=1}^{n}(y_i - (\hat{a} + \hat{b}x_i))^2}. \tag{8.5}$$

When $n = 1$ or 2 this expression is meaningless, which makes sense: with only two data points you can fit a line exactly and so there is no way of estimating error from the data alone.

More generally, in a regression with k predictors (that is, $y = X\beta + \epsilon$, with an $n \times k$ predictor matrix X), expression (8.5) becomes $\hat{\sigma} = \sqrt{\frac{1}{n-k}\sum_{i=1}^{n}(y_i - (X_i\hat{\beta}))^2}$, with $n - k$ in the denominator rather than n, adjusting for the k coefficients fit by least squares.

Computing the sum of squares directly

The least squares estimates of the coefficients can be computed directly using the formula in (8.2). But to develop understanding it can be helpful to write an R function to compute the sum of squares and then play around with different values of a and b.

First we write the function, which we call `rss` for "residual sum of squares":

```
rss <- function(x, y, a, b){     # x and y are vectors, a and b are scalars
  resid <- y - (a + b*x)
  return(sum(resid^2))
}
```

The above function is somewhat crude and is intended for home consumption and not production use, as we have not, for example, added lines to check that `x` and `y` are vectors of the same length, nor have we incorporated any methods for handling missing data (`NA` entries). Hence we are at the mercy of the R defaults if the function arguments are not specified exactly as desired. For exploration on our own, however, this simple function will do.

We can try it out: `rss(hibbs$growth, hibbs$vote, 46.3, 3.0)` evaluates the residual sum of squares at the least squares estimate, $(a, b) = (46.2, 3.1)$, and we can experiment with other values of (a, b) to check that it's not possible to get any lower with these data; see Exercise 7.1.

Maximum likelihood

If the errors from the linear model are independent and normally distributed, so that $y_i \sim \mathrm{N}(a+bx_i, \sigma^2)$ for each i, then the least squares estimate of (a, b) is also the maximum likelihood estimate. The *likelihood function* in a regression model is defined as the probability density of the data given the parameters and predictors; thus, in this example,

$$p(y|a, b, \sigma, X) = \prod_{i=1}^{n} \mathrm{N}(y_i|a + bx_i, \sigma^2), \tag{8.6}$$

where $\mathrm{N}(\cdot|\cdot, \cdot)$ is the normal probability density function,

$$\mathrm{N}(y \mid m, \sigma^2) = \frac{1}{\sqrt{2\pi}\sigma} \exp\left(-\frac{1}{2}\left(\frac{y-m}{\sigma}\right)^2\right). \tag{8.7}$$

A careful study of (8.6) reveals that maximizing the likelihood requires minimizing the sum of squared residuals; hence the least squares estimate $\hat{\beta} = (\hat{a}, \hat{b})$ can be viewed as a maximum likelihood estimate under the normal model.

There is a small twist in fitting regression models, in that the maximum likelihood estimate of σ is $\sqrt{\frac{1}{n}\sum_{i=1}^{n}(y_i - (\hat{a} + \hat{b}x_i))^2}$, without the $\frac{1}{n-2}$ adjustment given in (8.5).

In Bayesian inference, the uncertainty for each parameter in the model automatically accounts for the uncertainty in the other parameters. This property of Bayesian inference is particularly relevant for models with many predictors, and for advanced and hierarchical models.

Where do the standard errors come from? Using the likelihood surface to assess uncertainty in the parameter estimates

In maximum likelihood estimation, the likelihood function can be viewed as a hill with its peak at the maximum likelihood estimate.

Figure 8.1a displays the likelihood for a simple example as a function of the coefficients a and b. Strictly speaking, this model has three parameters—a, b, and σ—but for simplicity we display the likelihood of a and b conditional on the estimated $\hat{\sigma}$.

Figure 8.1b shows the maximum likelihood estimate $(\hat{a}, \hat{b}) = (46.2, 3.1)$. This is the value of the parameters where the likelihood function—the hill in Figure 8.1a—has its peak. Figure 8.1b also includes uncertainty bars showing ±1 standard error for each parameter. For example, the data are consistent with a being roughly in the range 46.2 ± 1.6 and with b being in the range 3.1 ± 0.7.

The likelihood function does not just have a maximum and a range; it also has a correlation. The area with highest likelihood surrounding the peak can be represented by an ellipse as is shown in Figure 8.1c. The shape of the uncertainty ellipse tells us something about the information in the data and model about the two parameters jointly. In this case the correlation is negative.

To understand this inferential correlation, see the scatterplot of the data from Figure 7.2 which we have reproduced in Figure 8.2a: the regression line goes through the cloud of points, most of which have positive values for x. Figure 8.2b shows a range of lines that are consistent with the data, with

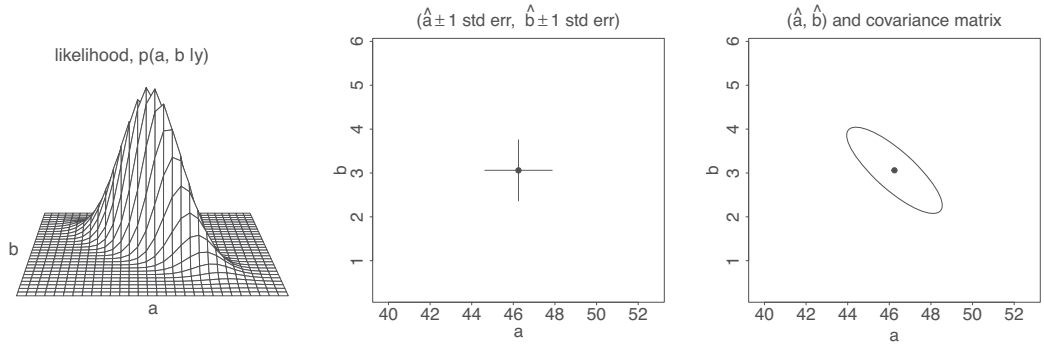


Figure 8.1 *(a) Likelihood function for the parameters a and b in the linear regression $y = a + bx +$ error, of election outcomes, y_i, on economic growth, x_i. (b) Mode of the likelihood function (that is, the maximum likelihood estimate $(\hat{a}, \hat{b})$) with ± 1 standard error bars shown for each parameter. (c) Mode of the likelihood function with an ellipse summarizing the inverse-second-derivative-matrix of the log likelihood at the mode.*

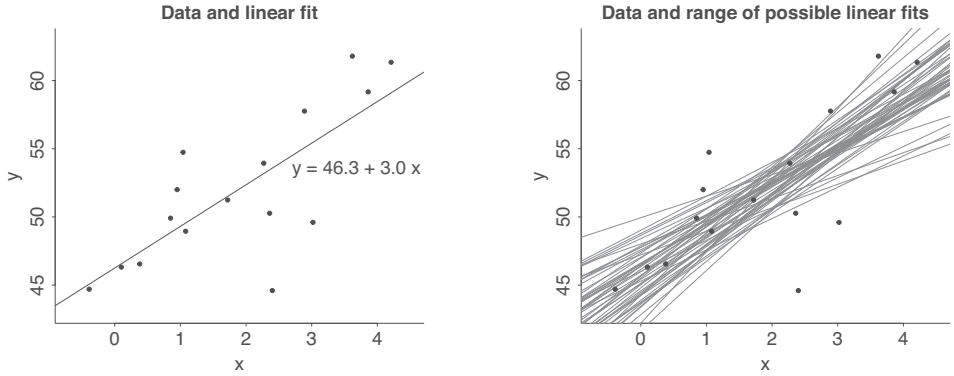


Figure 8.2 *(a) Election data with the linear fit, $y = 46.3 + 3.0x$, repeated from Figure 7.2b. (b) Several lines that are are roughly consistent with the data. Where the slope is higher, the intercept (the value of the line when $x = 0$) is lower; hence there is a negative correlation between a and b in the likelihood.*

the lines representing 50 draws from the Bayesian posterior distribution (see below). Lines of higher slope (for which b is higher) intersect the y-axis at lower values (and thus have lower values of a), and vice versa, hence the negative correlation in Figure 8.1c.

Bayesian inference

Least squares or maximum likelihood finds the parameters that best fit the data (according to some pre-specified criterion), but without otherwise constraining or guiding the fit. But, as discussed in Section 9.3 and elsewhere, we typically have prior information about the parameters of the model. Bayesian inference produces a compromise between prior information and data, doing this by multiplying the likelihood with a *prior distribution* that probabilistically encodes external information about the parameters. The product of the likelihood (in the above example, it is $p(y|a, b, \sigma)$ in (8.6), considered as a function of a, b, and σ) and the prior distribution is called the *posterior distribution* and it summarizes our knowledge of the parameter, *after* seeing the data. ("Posterior" is Latin for "later.")

In Chapter 9 we consider Bayesian inference from many angles. In the present section we focus on the posterior distribution as a modification of the likelihood function. The generalization of maximum likelihood estimation is maximum *penalized* likelihood estimation, in which the prior distribution is considered as a "penalty function" that downgrades the likelihood for less-favored

values of the parameter, again, thus giving an estimate that is typically somewhere in between the prior and what would be obtained by the data alone. Because of the anchoring of the prior, the maximum penalized likelihood estimate can be more stable than the raw maximum likelihood or least squares estimate.

In addition to adding prior information, Bayesian inference is also distinctive in that it expresses uncertainty using probability. When we fit a model using `stan_glm`, we obtain a set of simulation draws that represent the posterior distribution, and which we typically summarize using medians, median absolute deviations, and uncertainty intervals based on these simulations, as we shall discuss in Section 9.1.

We prefer to use Bayesian methods because of the flexibility of propagation of uncertainty using probability and simulation, and because inclusion of prior information can make inferences more stable. That said, it can be helpful to see the connections to least squares and maximum likelihood, as these simpler methods can be easier to understand and are overwhelmingly the most popular ways to fit regression in current practice. So in the following section we show how to fit models in both the classical and Bayesian frameworks.

Point estimate, mode-based approximation, and posterior simulations

The least squares solution is a *point estimate* that represents the vector of coefficients that provides the best overall fit to data. For a Bayesian model, the corresponding point estimate is the *posterior mode*, which provides the best overall fit to data and prior distribution. The least squares or maximum likelihood estimate is the posterior mode corresponding to the model with a uniform or flat prior distribution.

But we do not just want an estimate; we also want uncertainty. For a model with just one parameter, the uncertainty can be represented by the estimate ± standard error; more generally we use a bell-shaped probability distribution representing the multivariate uncertainty, as illustrated in Figure 8.1.

As discussed more fully in Chapter 9, it is convenient to summarize uncertainty using simulations from this mode-based approximation or, more generally, from the posterior distribution of the model parameters. By default, when we fit models using `stan_glm`, we get posterior simulations summarized by median and mad sd; see Section 5.3.

8.2 Influence of individual points in a fitted regression

From expressions (8.3) and (8.4), we can see that the least squares estimated regression coefficients $\hat{a}$ and $\hat{b}$ are linear functions of the data, y. We can use these linear expressions to understand the *influence* of each data point by looking at how much a change in each y_i would change $\hat{b}$. We could also work out the influence on the intercept—the predicted value when $x = 0$—or any other prediction under the model, but typically it is the slope that is most of interest.

From equation (8.3), we see that an increase of 1 in y_i corresponds to a change in $\hat{b}$ that is proportional to $(x_i - \bar{x})$:

- If $x_i = \bar{x}$, the influence of point i on the regression slope is 0. This makes sense: taking a point in the center and moving it up or down will affect the height of the fitted line but not its slope.
- If $x_i > \bar{x}$, the influence of point i is positive, with greater influence the further x_i is from the mean.
- If $x_i < \bar{x}$, the influence of point i is negative, with greater absolute influence the further x_i is from the mean.

One way to understand influence is to consider the fitted regression line as a rod attached to the data by rubber bands; then imagine how the position and orientation of the rod changes as individual data points are moved up and down. Figure 8.3 illustrates.

Influence can also be computed for multiple regression, using the matrix expression (equation

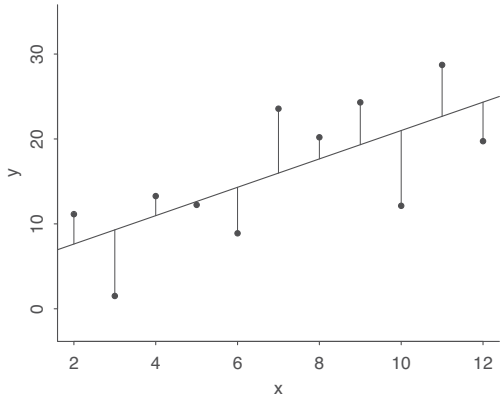


Figure 8.3 *Understanding the influence of individual data points on the fitted regression line. Picture the vertical lines as rubber bands connecting each data point to the least squares line. Take one of the points on the left side of the graph and move it up, and the slope of the line will* decrease*. Take one of the points on the right side and move it up, and the slope will* increase*. Moving the point in the center of the graph up or down will not change the slope of the fitted line.*

(8.2)), which reveals how the estimated vector of regression coefficients $\hat{\beta}$ is a linear function of the data vector y, and for generalized linear models, by re-fitting the regression after altering data points one at a time.

8.3 Least squares slope as a weighted average of slopes of pairs

In Section 7.3, we discussed that, when a regression $y = a + bx +$ error is fit with just an indicator variable (that is, where x just takes on the values 0 and 1), the least squares estimate of its coefficient b is simply the average difference in the outcome between the two groups; that is, $\bar{y}_1 - \bar{y}_0$.

There is a similar identity when the predictor x is continuous; in this case, we can express the estimated slope $\hat{b}$ from (8.3) as a weighted average of slopes.

The basic idea goes as follows. With n data points (x, y) there are n^2 pairs (including the possibility of taking the same data point twice). For each pair i, j we can compute the slope of the line connecting them:

$$\text{slope}_{ij} = \frac{y_j - y_i}{x_j - x_i}.$$

This expression is not defined when the two points have the same value of the predictor—that is, when $x_i = x_j$—but don't worry about that now; it will turn out that these cases drop out of our equation.

We would like to define the best-fit regression slope as an average of the individual slopes, but it makes sense to use a weighted average, in which slope_{ij} counts more if the two points are further apart in x. We might, then, weight each slope by the difference between the two values, $|x_j - x_i|$. For mathematical reasons that we do not discuss here but which relate to the use of the normal distribution for errors (which is in turn motivated by the Central Limit Theorem, as discussed in Section 3.5), it makes sense to weight each slope by the squared separation, $(x_j - x_i)^2$.

We can then compute the weighted average:

$$\begin{aligned}\text{weighted average of slopes} &= \frac{\sum_{i,j}(x_j - x_i)^2 \frac{y_j - y_i}{x_j - x_i}}{\sum_{i,j}(x_j - x_i)^2} \\ &= \frac{\sum_{i,j}(x_j - x_i)(y_j - y_i)}{\sum_{i,j}(x_j - x_i)^2}. \qquad (8.8)\end{aligned}$$

If you collect the terms carefully, you can show that this expression is the same as $\hat{b}$ in (8.3), so we can interpret the estimated coefficient $\hat{b}$ as the weighted average slope in the data, and we can interpret the underlying parameter b as the weighted average slope in the population.

8.4 Comparing two fitting functions: `lm` and `stan_glm`

The standard routine for fitting linear regressions in R is `lm`, which performs classical least squares regression and returns estimates and standard errors; indeed, that is what we used in our earlier book on regression, and there are comparable functions in all other statistical software. In this book, though, we use `stan_glm`, a program that performs Bayesian inference and returns estimates, standard errors, and posterior simulations.

We switch to `stan_glm` for two reasons. First, the simulations automatically computed by the new program represent uncertainty and can be used to obtain standard errors and predictive distributions for any function of current data, future data, and parameters. Second, the new program performs Bayesian inference, which can be used to get more stable estimates and predictions incorporating prior information. We discuss both these aspects of Bayesian inference—the probabilistic expression of uncertainty, and the inclusion of prior information—in Chapter 9, along with examples of how to perform probabilistic predictions and express prior information using regressions in `stan_glm`.

For many simple problems, classical and Bayesian inferences are essentially the same. For example, in Chapters 6 and 7, we fit regressions of earnings on incomes, daughters' height on mothers' heights, and elections on economic growth, in each case using `stan_glm` with its default settings. Switching to `lm` would give essentially identical results. The default prior is weak, hence it yields similar inferences to those obtained under maximum likelihood in these examples. The main role of the default prior in this sort of regression is to keep the coefficient estimates stable in some pathological cases such as near-collinearity; otherwise, it does not make much difference.

Bayesian inference makes more of a difference with weak data or strong priors. In addition, for advanced and hierarchical models, there are more differences between classical and Bayesian inferences: this is because more complex models can have latent or weakly identified parameters, so that a Bayesian inferential structure can provide smoother estimates more generally. In addition, advanced and multilevel models tend to increase in complexity as sample sizes increases, and so Bayesian inference can make a difference even with large datasets.

Reproducing maximum likelihood using `stan_glm` with flat priors and optimization

Here we demonstrate the commands for bridging between the Bayesian and classical estimates. Our default regression fit to some data frame `mydata` is

```
stan_glm(y ~ x, data=mydata)
```

When `stan_glm` is called without setting the prior arguments, it by default uses weak priors that partially pool the coefficients toward zero, as discussed in Section 9.5.

If we want to get a closer match to classical inference, we can use a flat prior, so that the posterior distribution is the same as the likelihood. Here is the function call:

```
stan_glm(y ~ x, data=mydata, prior_intercept=NULL, prior=NULL, prior_aux=NULL)
```

The three different `NULL`s set flat priors for the intercept, the other coefficients in the model, and σ, respectively.

Again, we explain all this in the next chapter; at this point we just want to demonstrate the function calls.

To move even closer to standard regression, we can tell Stan to perform optimization instead of sampling. This yields the maximum penalized likelihood estimate, which in the case of a flat prior is simply the maximum likelihood estimate:

```
stan_glm(y ~ x, data=mydata, prior_intercept=NULL, prior=NULL, prior_aux=NULL,
  algorithm="optimizing")
```

Running `lm`

As discussed in Section 1.6, if you only want to do maximum likelihood, and you have no interest in probabilistic predictions, you could instead fit regressions using the `lm` function in R:

```
lm(y ~ x, data=mydata)
```

In general, though, we prefer to use `stan_glm` even for maximum likelihood estimation because doing so allows us to propagate uncertainty in functions of parameters and data, as we shall explain in Chapter 9.

Confidence intervals, uncertainty intervals, compatibility intervals

In Section 4.2 we discussed the general idea of confidence intervals, which are a way of conveying inferential uncertainty. The starting point, based on the assumption of an unbiased, normally distributed estimate, is that an estimate ±1 standard error has an approximate 68% chance of including the true value of the quantity being estimated, and the estimate ±2 standard errors has an approximately 95% chance of covering the true value.

For linear regression, the residual standard deviation σ is itself estimated with error. If a regression with k coefficients is fit to n data points, there are said to be $n - k$ degrees of freedom (see page 53), and confidence intervals for regression coefficients are constructed using the t_{n-k} distribution. For example, the model $y = a + bx +$ error has 2 coefficients; if it is fit to 10 data points, then we can type `qt(0.975, 8)` in R, yielding the value 2.31, and so the 95% confidence interval for either of these coefficients is the estimate ±2.31 standard errors.

When fitting models using `stan_glm`, we can get approximate 68% and 95% intervals using median ±1 or 2 mad sd, or we can construct intervals directly from the simulations. Again, we shall discuss this further in Chapter 9, but, very briefly, the fit from `stan_glm` yields a matrix of simulations, each column of which corresponds to one of the parameters in the model. Here is a simple example:

```
x <- 1:10
y <- c(1,1,2,3,5,8,13,21,34,55)
fake <- data.frame(x, y)
fit <- stan_glm(y ~ x, data=fake)
print(fit)
```

which yields,

```
            Median MAD_SD
(Intercept) -13.8    6.7
x             5.1    1.1

Auxiliary parameter(s):
      Median MAD_SD
sigma 10.0    2.5
```

We then extract the simulations:

```
sims <- as.matrix(fit)
```

yielding a matrix `sims` with three columns, containing simulations of the intercept, the coefficient for x, and the residual standard deviation σ. To extract a 95% interval for the coefficient for x, we type `quantile(sims[,2], c(0.025, 0.975))` or `quantile(sims[,"x"], c(0.025, 0.975))`, either of which returns the interval [2.6, 7.5], which is not far from the approximation $[5.1 \pm 2 * 1.1]$ obtained using the median and mad sd.

All the confidence intervals discussed above can be called "uncertainty intervals," in that they represent uncertainty about the quantity being estimated, or "compatibility intervals," in that they give a range of parameter values that are most compatible with the observed data.

8.5 Bibliographic note

Various textbooks, for example Neter et al. (1996), derive least squares and maximum likelihood estimates and explain the relevant matrix algebra. Gelman and Greenland (2019) and Greenland (2019) discuss the ideas of "uncertainty intervals" and "compatibility intervals" as replacements for confidence intervals. Gelman et al. (2013) provide an introduction to Hamiltonian Monte Carlo and convergence diagnostics. See Stan Development Team (2020) for details of the algorithms implemented in `stan_glm` and R Core Team (2019) for background on `lm`.

8.6 Exercises

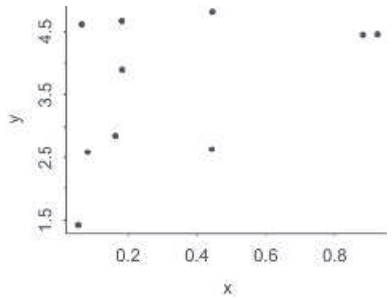
8.1 *Least squares*: The folder `ElectionsEconomy` contains the data for the example in Section 7.1. Load these data, type in the R function `rss()` from page 104, and evaluate it at several different values of (a, b). Make two graphs: a plot of the sum of squares of residuals as a function of a, with b fixed at its least squares estimate given in Section 7.1, and a plot of the sum of squares of residuals as a function of b, with a fixed at its least squares estimate. Confirm that the residual sum of squares is indeed minimized at the least squares estimate.

8.2 *Maximum likelihood*: Repeat the previous exercise but this time write a function, similar to `rss()` on page 104, that computes the logarithm of the likelihood (8.6) as a function of the data and the parameters a, b, σ. Evaluate this function as several values of these parameters, and make a plot demonstrating that it is maximized at the values computed from the formulas in the text (with σ computed using $\frac{1}{n}$, not $\frac{1}{n-2}$; see page 104).

8.3 *Least absolute deviation*: Repeat 8.1, but instead of calculating and minimizing the sum of squares of residuals, do this for the sum of absolute values of residuals. Find the (a, b) that minimizes the sum of absolute values of residuals, and plot the sum of absolute values of residuals as a function of a and of b. Compare the least squares and least absolute deviation estimates of (a, b).

8.4 *Least squares and least absolute deviation*: Construct a set of data $(x, y)_i$, $i = 1, \ldots, n$, for which the least squares and least absolute deviation (see Exercise 8.3) estimates of (a, b) in the fit, $y = a + bx$, are much different. What did you have to do to make this happen?

8.5 *Influence of individual data points*: A linear regression is fit to the data below. Which point has the most influence (see Section 8.2) on the slope?



8.6 *Influence of individual data points*:

(a) Using expression (8.3), compute the influence of each of the data points in the election

forecasting example on the fitted slope of the model. Make a graph plotting influence of point i vs. x_i.

(b) Re-fit the model n times, for each data point i adding 1 to y_i. Save $\hat{b}$ from each of these altered regressions, compare to the $\hat{b}$ from the original data, and check that the influence is approximately the same as computed above using the formula. (The two calculations will not give identical results because `stan_glm` uses a prior distribution and so it does not exactly yield the least squares estimate.)

8.7 *Least squares slope as a weighted average of individual slopes*:

(a) Prove that the weighted average slope defined in equation (8.8) is equivalent to the least squares regression slope in equation (8.3).

(b) Demonstrate how this works in a simple case with three data points, $(x, y) = (0, 0),\ (4, 1),\ (5, 5)$.

8.8 *Comparing* `lm` *and* `stan_glm`: Use simulated data to compare least squares estimation to default Bayesian regression:

(a) Simulate 100 data points from the model, $y = 2 + 3x +$ error, with predictors x drawn from a uniform distribution from 0 to 20, and with independent errors drawn from the normal distribution with mean 0 and standard deviation 5. Fit the regression of y on x data using `lm` and `stan_glm` (using its default settings) and check that the two programs give nearly identical results.

(b) Plot the simulated data and the two fitted regression lines.

(c) Repeat the two steps above, but try to create conditions for your simulation so that `lm` and `stan_glm` give much different results.

8.9 *Leave-one-out cross validation*: As discussed in the context of (8.5), the root mean square of residuals, $\sqrt{\frac{1}{n}\sum_{i=1}^{n}(y_i - (\hat{a} + \hat{b}x_i))^2}$, is an underestimate of the error standard deviation σ of the regression model, because of overfitting, that the parameters a and b are estimated from the same n data points as are being used to compute the residuals.

Cross validation, which we discuss in detail in Section 11.8, is an alternative approach to assessing predictive error that avoids some of the problems of overfitting. The simplest version of cross validation is the leave-one-out approach, in which the model is fit n times, in each case excluding one data point, fitting the model to the remaining $n - 1$ data points, and using this fitted model to predict the held-out observation:

- For $i = 1, \ldots, n$:
  - Fit the model $y = a + bx +$ error to the $n - 1$ data points $(x, y)_j,\ j \neq i$. Label the estimated regression coefficients as $\hat{a}_{-i}, \hat{b}_{-i}$.
  - Compute the cross-validated residual, $r_i^{\text{CV}} = y_i - (\hat{a}_{-i} + \hat{b}_{-i}x_i)$.
- Compute the estimate $\hat{\sigma}^{\text{CV}} = \sqrt{\frac{1}{n}\sum_{i=1}^{n} r_i^2}$.

(a) Perform the above steps for the elections model from Section 7.1. Compare three estimates of σ: (i) the estimate produced by `stan_glm`, (ii) formula (8.5), and (iii) $\hat{\sigma}^{\text{CV}}$ as defined above.

(b) Discuss any differences between the three estimates.

8.10 *Leave-one-out cross validation*: Create a fake dataset $(x, y)_i,\ i = 1, \ldots, n$, in such a way that there is a big difference between $\hat{\sigma}^{\text{CV}}$ as defined in the previous exercise, and the estimated residual standard deviation from (8.5). Explain what you did to create this discrepancy.

8.11 *Working through your own example*: Continuing the example from the final exercises of the earlier chapters, take your fitted regression line from Exercise 7.10 and compute the influence of each data point on the estimated slope. Where could you add a new data point so it would have zero influence on the estimated slope? Where could you add a point so it would have a large influence?